

UNIT III: 3.4 Views: Concept of View, CREATE VIEW Command, Updating Views, Views and Joins, Views and Sub Queries, Dropping Views



Prepared By
Mr. Mohammad Ali
Lecturer (Selection Grade)
M. H. Saboo Siddik Polytechnic, Mumbai

Objectives

After completing this lesson, you should be able to do the following:

- Describe a view.
- Create simple and complex views
- Alter the definition of views, and drop a view
- Retrieve data from a view
- Insert, update, and delete data through a view.

VIEWS

- In some cases, it is not desirable for all users to see the entire logical model.
- Consider a person who needs to know an employee's name, his designation and department name but has no need to see the salary and other attribute's values.
- This person should see a relation described, in SQL, by

```
select ename, job, dname  
from emp, dept  
where emp.deptno = dept.deptno )
```

- **A View is a "Virtual Table".**
- **It is not like a simple table, but is a virtual table which contains columns and data from different tables (may be one or more tables).**
- **The tables referred in the views are known as Base Tables.**
- **Creating a view does not take any storage space as only the query is stored in the data dictionary and the actual data is fetched from Base Table.**

VIEWS

EMP Table							
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20

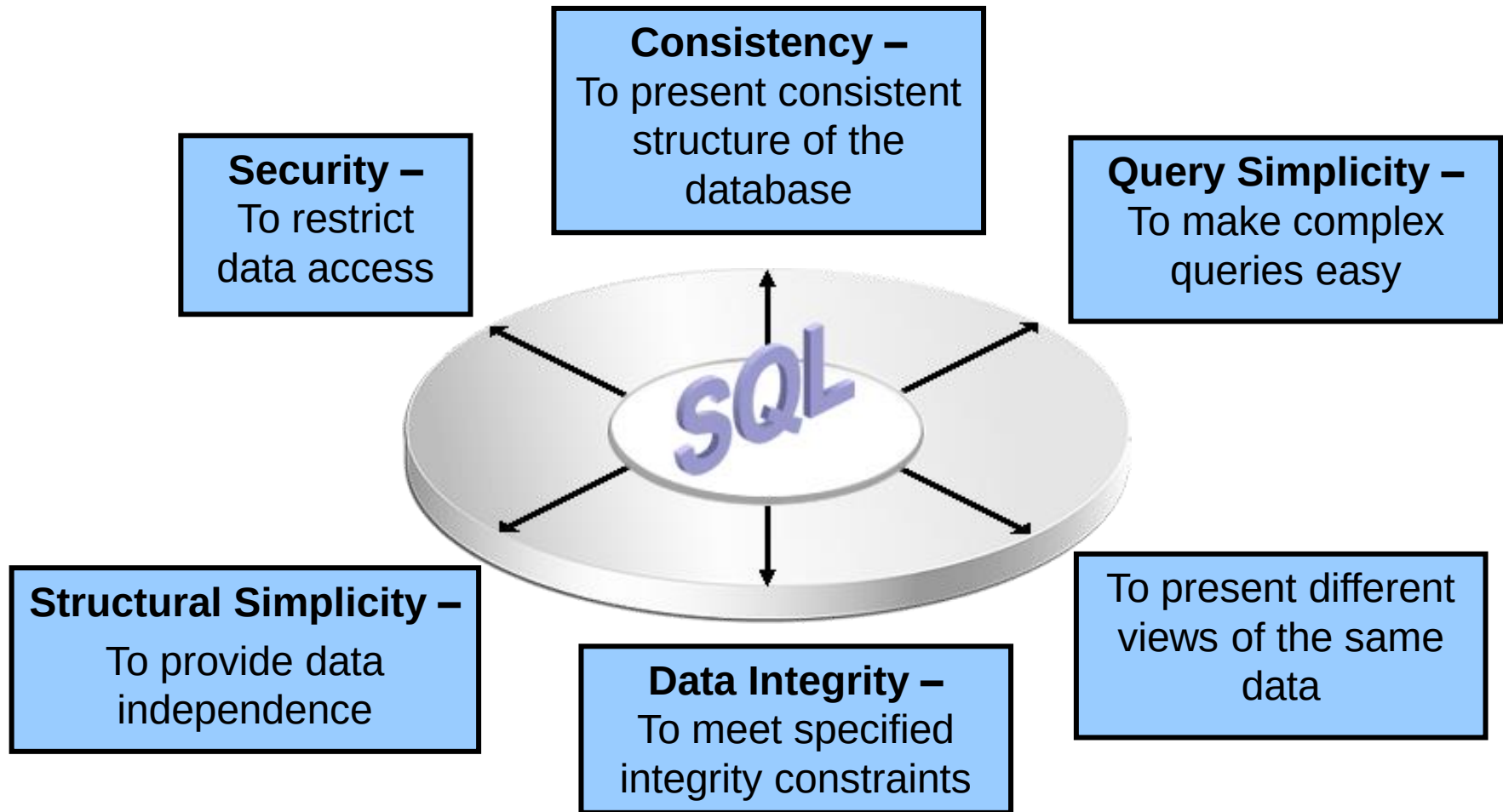
DEPT Table		
DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

ENAME	JOB	DNAME
SMITH	CLERK	RESEARCH
ALLEN	SALESMAN	SALES
WARD	SALESMAN	SALES
JONES	MANAGER	RESEARCH

View logically represents subsets of data from one or more tables

View is created from EMP(ENAME & JOB columns) and DEPT(DNAME column) tables.

Advantages of Views



Simple Views and Complex Views

Feature	Simple Views	Complex Views
Number of tables	One	One or more
Contain functions	No	Yes
Contain groups of data	No	Yes
DML operations through a view	Yes	Not always

Creating a View

- Database views are created using the **CREATE VIEW** statement. Views can be created from a single table, multiple tables, or another view.
- To create a view, a user must have the appropriate system privilege according to the specific implementation.
- The basic CREATE VIEW syntax is as follows:

```
CREATE VIEW view_name AS  
SELECT column1, column2.....  
FROM table_name  
WHERE [condition];
```

Creating a View

- We embed a subquery in the `CREATE VIEW` statement:

```
CREATE [OR REPLACE] [FORCE|NOFORCE] VIEW <view_name>
    [(alias[, alias]...)]
    AS subquery
    [WITH CHECK OPTION [CONSTRAINT constraint]]
    [WITH READ ONLY [CONSTRAINT constraint]];
```

- The subquery can contain complex `SELECT` syntax.

- **OR REPLACE** - recreates the view if it already exists.
- **FORCE** - creates the views regardless of whether or not the base element exist.
- **NOFORCE** - creates the view only if the base table exist.
- **view_name** - is the name of the view.
- **alias** - specifies names for the expression selected by the view's query.
- **subquery** - is a complete `SELECT` statement.
- **WITH CHECK OPTION** - specifies only the rows accessible to the view can be inserted or updated.
- **constraint** - is the name assigned to the `CHECK OPTION` constraint.
- **WITH READ ONLY** - Ensures that no DML operations can be performed on this view.

Creating a View

- Create the **EMPVU30** view, which contains details of employees in department 30:

```
CREATE VIEW empvu30
AS SELECT empno, ename, job, sal, deptno
FROM      emp
WHERE     deptno = 30;
```

```
SQL> CREATE VIEW empvu30
  2 AS SELECT empno, ename, job, sal, deptno
  3 FROM      emp
  4 WHERE     deptno = 30;

View created.
```

- Describe the structure of the view by using the **DESCRIBE** command:

```
DESCRIBE empvu30;
```

```
SQL> describe empvu30;
```

Name	Null?	Type
EMPNO	NOT NULL	NUMBER(4)
ENAME		VARCHAR2(10)
JOB		VARCHAR2(9)
SAL		NUMBER(7,2)
DEPTNO		NUMBER(2)

Creating a View

- Create the **SALVU30** view, by using column aliases in the subquery:

```
CREATE VIEW salvu30
AS SELECT empno EMP_ID,
          ename NAME,
          sal*12 ANNUAL_SAL
FROM emp
WHERE deptno = 30;
```

```
SQL> CREATE VIEW salvu30
  2 AS SELECT empno EMP_ID, ename NAME,
  3           sal*12 ANNUAL_SAL
  4 FROM emp
  5 WHERE deptno = 30;

View created.
```

- Retrieve Data by selecting the columns from this view by the given alias names and select everything:

```
SELECT emp_id, name, annual_sal
FROM salvu30;
```

```
SELECT *
FROM salvu30;
```

```
SQL> SELECT emp_id, name, annual_sal
  2 FROM salvu30;
```

EMP_ID	NAME	ANNUAL_SAL
7499	ALLEN	19200
7521	WARD	15000
7654	MARTIN	15000
7698	BLAKE	34200
7844	TURNER	18000
7900	JAMES	11400

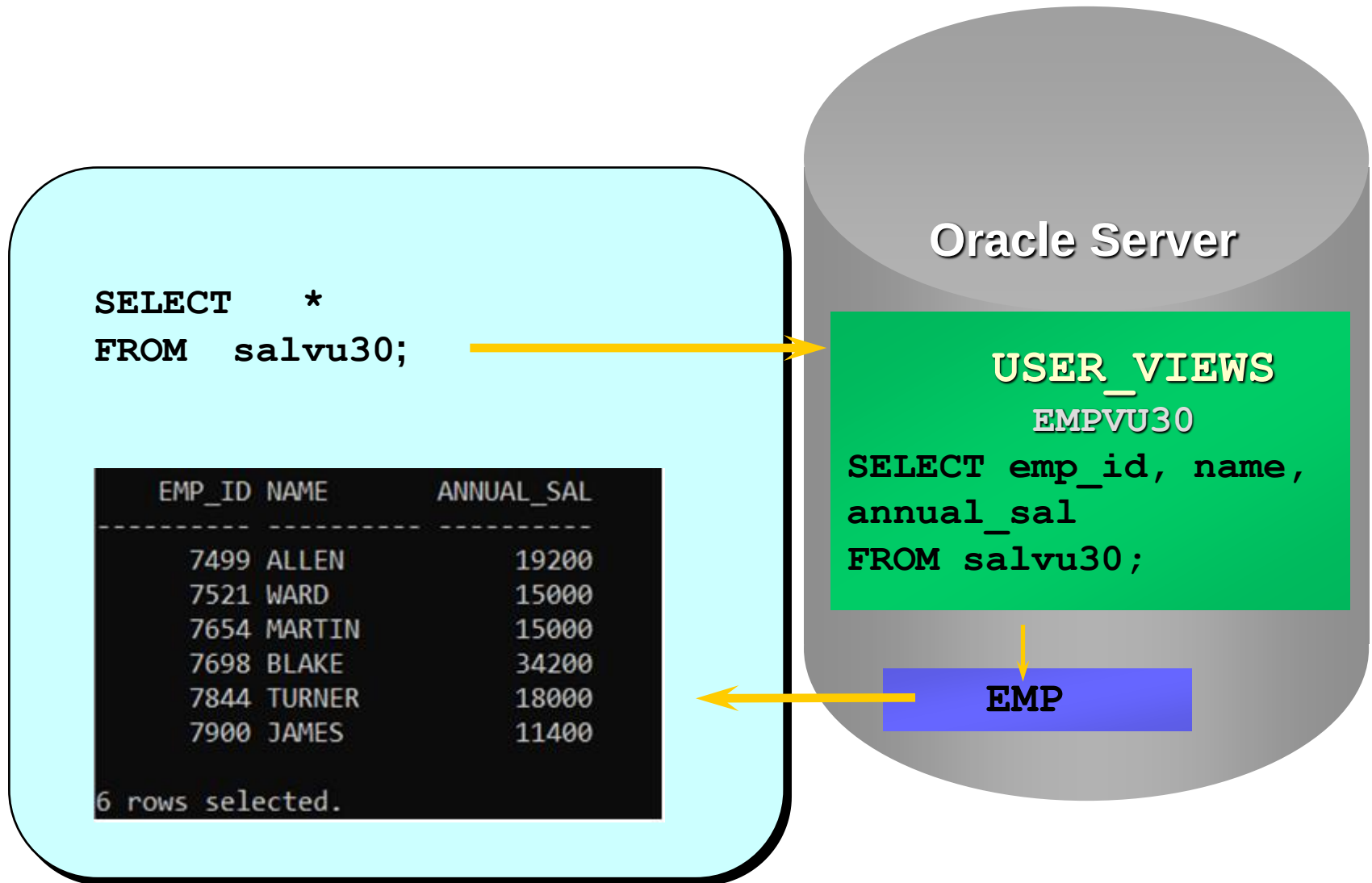
6 rows selected.

```
SQL> select * from salvu30;
```

EMP_ID	NAME	ANNUAL_SAL
7499	ALLEN	19200
7521	WARD	15000
7654	MARTIN	15000
7698	BLAKE	34200
7844	TURNER	18000
7900	JAMES	11400

6 rows selected.

Querying a View



Modifying a View

- Modify the **EMPVU30** view by using a **CREATE OR REPLACE VIEW** clause. Add an alias for each column name:

```
CREATE OR REPLACE VIEW empvu30
(emp_id, name, position, salary, total_sal, dept_id, mgr_id)
AS SELECT empno, ename, job, sal, sal+comm, deptno, mgr
FROM emp
WHERE deptno = 30;
```

View created.

- Column aliases in the **CREATE OR REPLACE VIEW** clause are listed in the same order as the columns in the subquery.

```
SELECT * FROM empvu30;
```

```
SQL> select * from empvu30;
```

EMP_ID	NAME	POSITION	SALARY	TOTAL_SAL	DEPT_ID	MGR_ID
7499	ALLEN	SALESMAN	1600	1900	30	7698
7521	WARD	SALESMAN	1250	1750	30	7698
7654	MARTIN	SALESMAN	1250	2650	30	7698
7698	BLAKE	MANAGER	2850		30	7839
7844	TURNER	SALESMAN	1500	1500	30	7698
7900	JAMES	CLERK	950		30	7698

```
6 rows selected.
```

Inserting into EMPVU30

```
SQL> DESC EMPVU30;
```

Name	Null?	Type
EMP_ID	NOT NULL	NUMBER(4)
NAME		VARCHAR2(10)
POSITION		VARCHAR2(9)
SALARY		NUMBER(7,2)
TOTAL_SAL		NUMBER
DEPT_ID		NUMBER(2)
MGR_ID		NUMBER(4)

```
SQL> SELECT * FROM EMPVU30;
```

EMP_ID	NAME	POSITION	SALARY	TOTAL_SAL	DEPT_ID	MGR_ID
7499	ALLEN	SALESMAN	1600	1900	30	7698
7521	WARD	SALESMAN	1250	1750	30	7698
7654	MARTIN	SALESMAN	1250	2650	30	7698
7698	BLAKE	MANAGER	2850		30	7839
7844	TURNER	SALESMAN	1500	1500	30	7698
7900	JAMES	CLERK	950		30	7698

```
6 rows selected.
```

```
SQL> INSERT INTO EMPVU30 VALUES(8001, 'ANAM', 'ASST. MANAGER', 5000, 30, 7698);
INSERT INTO EMPVU30 VALUES(8001, 'ANAM', 'ASST. MANAGER', 5000, 30, 7698)
```

```
*
```

```
ERROR at line 1:
```

```
ORA-00947: not enough values
```

```
SQL> INSERT INTO EMPVU30 VALUES(8001, 'ANAM', 'ASST. MANAGER', 5000, 5500, 30, 7698);
INSERT INTO EMPVU30 VALUES(8001, 'ANAM', 'ASST. MANAGER', 5000, 5500, 30, 7698)
```

```
*
```

```
ERROR at line 1:
```

```
ORA-01733: virtual column not allowed here
```

Creating a Complex View

- Create a complex view that contains group functions to display values from two tables:

```
CREATE VIEW dept_sal_vu
(dept_name, totalsal, minsal, maxsal, avgsal)
AS SELECT  d.dname, SUM(e.sal), MIN(e.sal),
           MAX(e.sal), AVG(e.sal)
FROM    emp e, dept d
WHERE   e.deptno = d.deptno
GROUP BY d.dname;
```

View created.

```
SELECT * FROM dept_sal_vu;
```

```
SQL> select * from dept_sal_vu;
```

DEPT_NAME	TOTALSAL	MINSAL	MAXSAL	AVGSAL
ACCOUNTING	8750	1300	5000	2916.66667
RESEARCH	10875	800	3000	2175
SALES	9400	950	2850	1566.66667

Inserting, deleting and updating a Complex View

```
SQL> insert into dept_sal_vu
  2  values('MARKETING', 500000, 50000, 60000, 55000);
insert into dept_sal_vu
*
ERROR at line 1:
ORA-01779: cannot modify a column which maps to a non key-preserved table
```

```
SQL> delete from dept_sal_vu
  2  where dept_name='SALES';
delete from dept_sal_vu
*
ERROR at line 1:
ORA-01732: data manipulation operation not legal on this view
```

```
SQL> update dept_sal_vu
  2  set totalsal=50000
  3  where dept_name = 'SALES';
update dept_sal_vu
*
ERROR at line 1:
ORA-01732: data manipulation operation not legal on this view
```

```
SQL> describe dept_sal_vu;
```

Name	Null?	Type
DEPT_NAME		VARCHAR2(14)
TOTALSAL		NUMBER
MINSAL		NUMBER
MAXSAL		NUMBER
AVGSAL		NUMBER

Rules for Performing DML Operations on a View

- We can usually perform DML operations on simple views. 
- We cannot remove a row if the view contains the following: 
 - Group functions
 - A `GROUP BY` clause
 - The `DISTINCT` keyword
 - The pseudocolumn `ROWNUM` keyword

Rules for Performing DML Operations on a View

You cannot modify data in a view if it contains:

- Group functions
- A `GROUP BY` clause
- The `DISTINCT` keyword
- The pseudocolumn `ROWNUM` keyword
- Columns defined by expressions



Rules for Performing DML Operations on a View

We cannot add data through a view if the view includes:

- Group functions
- A `GROUP BY` clause
- The `DISTINCT` keyword
- The pseudocolumn `ROWNUM` keyword
- Columns defined by expressions
- `NOT NULL` columns in the base tables that are not selected by the view



Performing DML Operations on a View

```
SQL> select * from grades;
```

SNAME	C_CODE	MARKS
ANAM	DMS	68
ANAM	CGR	64
JUNAID	DMS	66
JUNAID	CGR	67
HABBAN	DMS	70
HABBAN	CGR	65
CHAITANYA	DMS	60
CHAITANYA	CGR	62

8 rows selected.

```
SQL> CREATE VIEW DMS_MARKSVU AS SELECT * FROM GRADES WHERE C_CODE='DMS';
```

View created.

```
SQL> SELECT * FROM DMS_MARKSVU;
```

SNAME	C_CODE	MARKS
ANAM	DMS	68
JUNAID	DMS	66
HABBAN	DMS	70
CHAITANYA	DMS	60

Performing DML Operations on a View

```
SQL> INSERT INTO DMS_MARKSVU VALUES('CHETAN', 'DMS', 67);

1 row created.

SQL> SELECT * FROM DMS_MARKSVU;

SNAME          C_CODE          MARKS
-----
ANAM            DMS              68
JUNAID          DMS              66
HABBAN          DMS              70
CHAITANYA       DMS              60
CHETAN          DMS              67

SQL> UPDATE DMS_MARKSVU SET MARKS=65 WHERE SNAME='CHETAN';

1 row updated.

SQL> SELECT * FROM DMS_MARKSVU;

SNAME          C_CODE          MARKS
-----
ANAM            DMS              68
JUNAID          DMS              66
HABBAN          DMS              70
CHAITANYA       DMS              60
CHETAN          DMS              65

SQL> DELETE FROM DMS_MARKSVU WHERE SNAME='CHETAN';

1 row deleted.

SQL> SELECT * FROM DMS_MARKSVU;

SNAME          C_CODE          MARKS
-----
ANAM            DMS              68
JUNAID          DMS              66
HABBAN          DMS              70
CHAITANYA       DMS              60
```

Using the **WITH CHECK OPTION** Clause

- We can ensure that DML operations performed on the view stay in the domain of the view by using the **WITH CHECK OPTION** clause:

```
CREATE OR REPLACE VIEW empvu20
AS SELECT *
FROM emp
WHERE deptno = 20
WITH CHECK OPTION CONSTRAINT empvu20 ck ;
```

View created.

- Any attempt to change the department number for any row in the view fails because it violates the **WITH CHECK OPTION** constraint.

Denying DML Operations

- We can ensure that no DML operations occur by adding the **WITH READ ONLY** option to our view definition.
- Any attempt to perform a DML operation on any row in the view results in an Oracle server error.

```
CREATE OR REPLACE VIEW empvu10
(employee_number, employee_name, job_title)
AS SELECT empno, ename, job
FROM emp
WHERE deptno = 10
WITH READ ONLY;
View created.
```

Removing a View

We can remove a view without losing data because a view is based on underlying tables in the database.

```
DROP VIEW <view_name>;
```

```
DROP VIEW empvu30;  
View dropped.
```

Inline Views / Sub Queries

- An inline view is a **subquery** with an alias that we can use within a SQL statement.
- **An *inline view* is basically** a query inside another query, which makes it a subquery. However, an *inline view* is a subquery with a twist. It only exists inside of the FROM clause.
- A **named subquery** in the **FROM** clause of the main query is an example of an inline view.
- Most common usage – “TOP-N” analysis.
- Inline views are commonly used to simplify complex queries by removing join operations and condensing several queries into a single query.
- An inline view is not a schema object.

Top-N Analysis

Top-N queries ask for the n largest or smallest values of a column. For example:

- What are the ten best selling products?
- What are the ten worst selling products?

Both largest values and smallest values sets are considered Top-N queries.

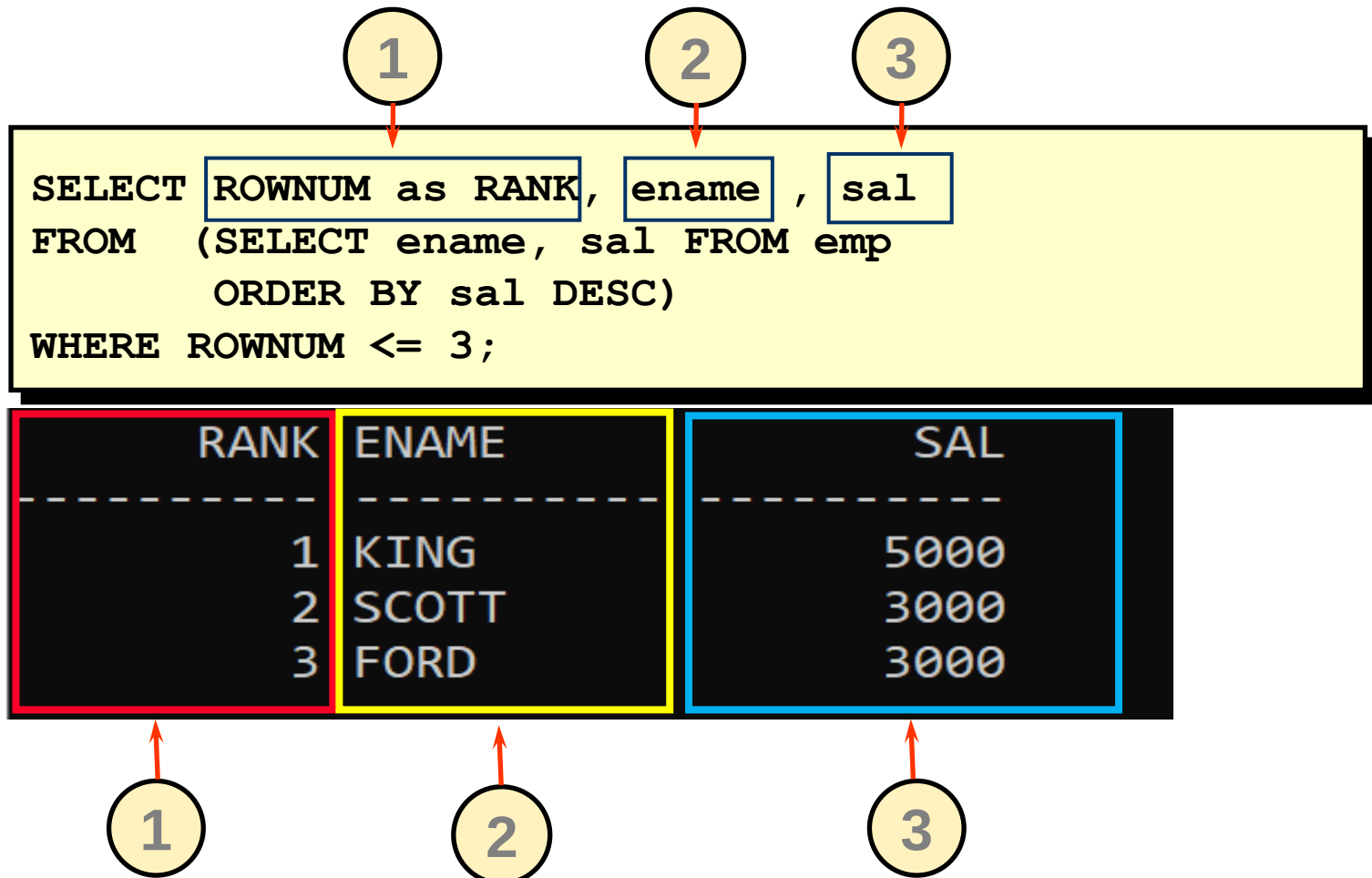
- The top-n-analysis query uses an inline subquery to return a result set.
- We can use ROWNUM in our query to assign a row number to the result set.
- The main query then uses ROWNUM to order the data and return the top n-rows

The high-level structure of a Top-N analysis query is:

```
SELECT  [column_list], ROWNUM
FROM    (SELECT [column_list]
          FROM   table
          ORDER  BY Top-N_column)
WHERE   ROWNUM <= N;
```

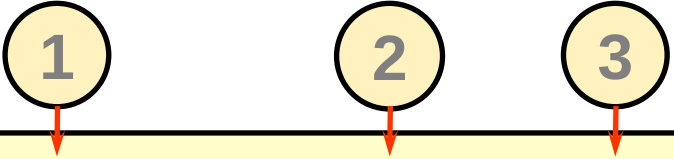
Example of Top-N Analysis

To display the top three earner names and salaries from the **EMP** table:



Example of Top-N Analysis

To display the top n earner names and salaries from the **EMP** table:



```
SELECT ROWNUM as RANK, ename, sal
FROM (SELECT ename, sal FROM emp
      ORDER BY sal DESC)
```

RANK	ENAME	SAL
1	KING	5000
2	FORD	3000
3	SCOTT	3000
4	JONES	2975
5	BLAKE	2850
6	CLARK	2450
7	ALLEN	1600
8	TURNER	1500
9	MILLER	1300
10	WARD	1250
11	MARTIN	1250
12	ADAMS	1100
13	JAMES	950
14	SMITH	800

14 rows selected.

Summary

- In this lesson, we have learned that a view is derived from data in other tables or views and provides the following advantages:
 - Restricts database access
 - Simplifies queries
 - Provides data independence
 - Provides multiple views of the same data
 - Can be dropped without removing the underlying data
 - An inline view is a subquery with an alias name.
 - Top-N analysis can be done using subqueries and outer queries.

A large, stylized graphic featuring the words "Question & Answer" in a bold, sans-serif font. The word "Question" is in blue, and the ampersand "&" is in yellow. The word "Answer" is also in blue. The text is centered within a large, horizontal oval shape. The oval has a thick yellow border with a blue outline. The background is white.

Question & Answer